Compute4Me Study Notes — Section 7: Serverless and Cloud-Native Distributed ML

> **Context:** Earlier sections focused on classical PS/all-reduce, Ray, elastic/heterogeneous training, decentralized/volunteer DL, and federated/privacy-aware paradigms. This section looks at **serverless and cloud-native distributed ML**: Databricks serverless, AWS Lambda-style serverless ML, and Ray-on-Kubernetes/managed clouds. The goal is to understand what these platforms do, and how Compute4Me differs.

7.1 What "Serverless" Means in ML Context

In ML, **serverless** usually means:

- You **do not provision or manage VMs or clusters**.
- You submit code or a container; the cloud provider automatically:
  - Allocates resources when needed.
  - Scales them up/down.
  - Bills you per-use.

Common cases:

- **Serverless ML inference:** Running models as functions (AWS Lambda, Azure Functions, GCP Cloud Functions).
- **Serverless distributed ML:** Platforms like Databricks serverless compute that spin up distributed clusters on demand for training/tuning.

For Compute4Me, the idea is similar but over **user-owned machines** instead of hidden cloud infrastructure.

7.2 Databricks Serverless Distributed ML

7.2.1 What Databricks Provides

Databricks has added support for **distributed ML on both serverless and standard clusters**: [web:24] [web:142]

- Blog (2025): *"Announcing the Public Preview of Distributed ML on Serverless and Standard Clusters"* https://www.databricks.com/blog/announcing-public-preview-distributed-ml-serverless-and-standard-clusters

Key features from the blog :[web:24]

- Run **distributed ML workloads** on serverless and standard clusters, including:
  - Training models with Apache Spark MLlib (Python).
  - Large-scale hyperparameter tuning with Optuna.
  - Experiment tracking with MLflow Spark.
  - Distributed single-node ML workloads (e.g., scikit-learn, XGBoost, LightGBM) via Joblib over Spark.
- **Unified compute and governance**:
  - Lakeguard + Spark Connect provide fine-grained access control (FGAC), multi-user isolation, and security.
- **Serverless benefits**:
  - Instant scale-up/down.
  - No idle cluster costs (pay for usage).
  - Simplified operations (no cluster management).

Azure Databricks has similar serverless compute:[web:147]

- Docs: *"Connect to serverless compute"* https://learn.microsoft.com/en-us/azure/databricks/compute/serverless/

7.2.2 How It Works Conceptually

Under the hood:

- Databricks runs **managed clusters** behind the scenes.
- When you choose "serverless" compute:
  - Your notebook/job attaches to a logical cluster type.
  - Databricks provisions containers/VMs in its own cloud account.
  - It handles autoscaling, fault tolerance, security.

For distributed ML:

- Spark MLlib, Optuna, MLflow, and Joblib Spark use Spark's distributed runtime to parallelize across machines.

You never see the underlying VMs.

7.2.3 Difficulty to Use vs Reimplement

- **Using Databricks serverless**:
  - Easy from a user perspective (pick serverless cluster in UI, run ML code).
  - But you must be on Databricks; this is proprietary.
- **Reimplementing Databricks-style serverless**:
  - Very hard: requires building a full multi-tenant, secure, autoscaling cluster service, plus ML orchestration on top.

7.2.4 Relevance to Compute4Me

Databricks serverless shows:

- It's valuable to **hide infrastructure complexity** while still supporting distributed ML.
- Governance and access control are first-class concerns in multi-tenant ML.

Compute4Me differs:

- Databricks: serverless on **cloud VMs** under one provider's control.
- Compute4Me: serverless-like UX on **volunteer/user-owned machines**.

You can mimic the UX (simple job submission, no cluster thinking) but your control plane and security model are completely different.

7.3 Serverless ML Inference (AWS Lambda and Friends)

7.3.1 Function-as-a-Service (FaaS) for ML

FaaS platforms (AWS Lambda, Azure Functions, GCP Cloud Functions) let you deploy **small ML models** as functions:

- You upload a function (possibly as a Docker image).
- The provider runs it on demand in response to events (HTTP requests, messages, etc.).
- You pay only for compute time and memory used.

Limitations:[web:39] [web:143] [web:148]

- Execution time limits (e.g., 15 minutes on AWS Lambda).[web:143]
- Limited memory and disk.
- Stateless by default (models must be reloaded per cold start unless cached).

7.3.2 Example: Deep Learning Inference with AWS Lambda

- AWS blog: *"Building deep learning inference with AWS Lambda and Amazon EFS"* https://aws.amazon.com/blogs/compute/building-deep-learning-inference-with-aws-lambda-and-amazon-efs/[web:148]

Key patterns:[web:39] [web:148]

- Use **Lambda + EFS** (network file system) to store large models and libraries.
- Package dependencies in Docker images.
- Use TF Lite or ONNX to reduce model size.

Tutorials:

- "Serverless Deep Learning: From Notebook to Production with AWS Lambda" (example on dev.to).

https://dev.to/austin_deyan_6c9b2445aed6/serverless-deep-learning-from-notebook-to-production-with-aws-lambda-3386[web:140]

- "Serverless ML pipeline with AWS Lambda" (Better Dev). https://betterdev.blog/serverless-ml-on-aws-lambda/[web:39]

7.3.3 Difficulty to Use vs Reimplement

- Using serverless for inference: medium — requires understanding of FaaS limits, packaging dependencies, cold start optimization.
- Reimplementing FaaS: extremely hard; you'd rely on existing providers.

7.3.4 Relevance to Compute4Me

Compute4Me is focused more on **training and heavy batch inference** than microsecond-scale API inference. Still, serverless ML shows:

- How to package ML workloads into containers/functions with strict resource limits.
- Patterns for **on-demand scaling** and pay-per-use pricing.

You can borrow ideas for:

- Packaging worker containers.
- Supporting "job-as-a-function" semantics for some workloads.

7.4 Ray on Kubernetes and Managed Clouds

7.4.1 Ray + Kubernetes

Ray is increasingly integrated with Kubernetes and managed services:[web:144][web:152]

- Google Cloud blog: *"Ray on GKE: New features for AI scheduling and scaling"* https://cloud.google.com/blog/products/containers-kubernetes/ray-on-gke-new-features-for-ai-scheduling-and-scaling[web:152]
- IBM Cloud: *"Ray on IBM Cloud Code Engine: Boost Your Serverless Compute"* https://www.ibm.com/new/product-blog/ray-on-ibm-cloud-code-engine[web:146]

Key features (Ray on GKE):[web:144][web:152]

- **Label-based scheduling**: assign labels to nodes (e.g., `gpu-family=L4`, `market-type=spot`) and choose nodes for tasks and actors accordingly.
- **Device Resource Allocation (DRA) for accelerators**: better sharing of GPUs.
- **Vertical pod resizing and writable cgroups**: improved resource utilization.
- Ray's autoscaler integrates with Kubernetes to scale pods based on workload.

IBM Cloud Code Engine example:[web:146]

- Code Engine provides a **serverless container platform**.

- Ray runs inside Code Engine's namespace; Code Engine handles scaling and infrastructure.

7.4.2 Difficulty to Use vs Reimplement

- Using Ray on K8s:
  - Medium–hard: requires Kubernetes familiarity, Helm charts or Ray Operator.
- Reimplementing something like Ray+K8s: very hard; heavy infra engineering.

7.4.3 Relevance to Compute4Me

Ray+K8s is essentially **cloud-native Compute4Me** for enterprises:

- Nodes are VMs managed by cloud + Kubernetes.
- Scheduling uses labels, autoscaling, and K8s primitives.

Compute4Me differs:

- Nodes are **not** in a single K8s cluster; they are random Ubuntu hosts behind NAT.
- You cannot rely on shared K8s control plane; must build your own discovery and scheduling across disparate machines.

Still, you can reuse ideas:

- Label-like attributes on worker nodes (e.g., `gpu=RTX3070`, `vram_gb=8`, `net=slow`).
- Placement policies similar to Ray label selectors when matching jobs to nodes.

7.5 How Serverless/Cloud-Native ML Shapes Compute4Me

7.5.1 UX Lessons

From Databricks serverless and Ray-on-cloud:

- **Hide cluster details** from end users: they submit jobs or attach notebooks, not manage nodes.
- Provide **simple, declarative job configs** (e.g., YAML/JSON) specifying:
  - GPUs needed, memory, expected duration.
  - Data sources.
  - Privacy mode (central vs federated).

Compute4Me can mirror this UX, even if it runs on volunteers.

7.5.2 Technical Lessons

- Autoscaling is central: cloud systems automatically scale clusters; Compute4Me should automatically recruit/retire volunteer nodes.
- Multi-tenant isolation and governance are crucial in cloud; in Compute4Me, the analogy is **sandboxing tasks** (container isolation, resource limits).

7.5.3 Clear Differentiation

Serverless and cloud-native ML answer: "How do I avoid managing infrastructure in the cloud?"

Compute4Me answers: "How do I **create infrastructure out of people's spare machines** and still offer a cloud-like UX?"

Key differences:

- **Control:** Cloud provider fully controls physical nodes; Compute4Me doesn't.
- **Trust:** Cloud nodes are trusted; Compute4Me nodes are semi-trusted or untrusted.
- **Topology:** Cloud nodes are on fast data center networks; Compute4Me nodes are scattered across the internet.

Thus, your novelty lies not in re-creating Databricks or Lambda, but in **adapting serverless UX patterns** to a completely different substrate.

7.6 Summary

This section shows that major vendors (Databricks, AWS, Azure, GCP, IBM) have converged on **serverless and cloud-native patterns for ML**:

- Databricks serverless and similar offerings provide **distributed ML without visible clusters**, relying on managed cloud resources.[web:24] [web:147]
- Serverless FaaS platforms make **model inference** elastic and pay-per-use, but with strict limits.[web:39] [web:140] [web:148]
- Ray-on-Kubernetes integrates a powerful AI compute engine with a cloud-native control plane.[web:144] [web:152] [web:146]

Compute4Me can treat these as **UX and architectural inspirations**, but its differentiator is the **resource pool**: user-owned, heterogeneous, unreliable machines rather than centrally managed cloud clusters.

This concludes the cloud/serverless background portion. The next step is to synthesize Sections 2–7 into a concrete architecture and roadmap for Compute4Me.